NOVA ONLINE BANK

# Nova Bank Institution Integration API

Version 1.0.0 · OpenAPI 3.0.3 · Nova Sync 101.1

**Nova Online Bank** — REST API for banks, payment institutions, card acquirers, and fintech partners.

Protocol version **Nova Sync 101.1** · Visa Card Authorization 101.1 · SWIFT / IBAN rails · Settlement middleware.

## Authentication
| Scheme | Header | Use |
|---|---|---|
| Member JWT | `Authorization: Bearer <token>` | End-user / member operations |
| Admin | `X-Admin-Key` | Ledger import, settlement run, funding seed |
| Merchant | `X-Merchant-Key` or `X-Nova-Merchant-Key` | Card pre-auth & authorize (merchant API) |
| Webhook | `X-Nova-Signature` | Inbound provider webhooks |
| Institution | `X-Institution-Api-Key` + `X-Client-Cert-Fingerprint` | mTLS institution API (wires, balances, inbound credit, LP sender) |

## Required headers (member flows)
- `X-Nova-Protocol: 101.1` — Nova Sync routing (online m0 / offline m1)
- `Idempotency-Key` — required on all `POST` mutations marked idempotent

## Integration entry points
1. **Discovery** — `GET /global/status`, `GET /global/integrations`, `GET /international/integrations`
2. **Popular rail** — `POST /bank/rail/send` (account / IBAN+BIC SEPA or SWIFT / crypto) and `GET /bank/rail/receive`. Dispatches to `POST /fiat/send` and `POST /crypto/withdraw`. Not AI Builder.
3. **Send fiat** — `POST /fiat/send` (Nova Bank API: ledger or SEPA/SWIFT). Canonical member fiat send. `POST /external/transfer` and `POST /transfers/by-party` remain.
4. **Party resolve** — `GET /accounts/resolve` (account # / IBAN) · `GET /external/resolve/{iban}` (thin IBAN wrapper)
5. **Send by party** — `POST /transfers/by-party` (internal account # or external IBAN+BIC — never email)
6. **Inbound credit** — `POST /institutional/inbound/credit` (institution) · `POST /inbound/credit` (admin) · `POST /bank/rail/inbound` · `POST /ledger/import/admin/all-to-account`
7. **International fiat send** — `POST /international/send` (IBAN + BIC → SEPA/SWIFT via `/fiat/send`). Local Ledger auto-promotes to Production. `POST /international/route` remains discovery-only.
8. **Settlement** — `GET /ledger-bridge/status`, `POST /ledger-bridge/settle`
9. **Cards** — merchant pre-auth under `/cards/merchant/*`
10. **Visa JIT** — `POST /visa/marqeta/jit-funding` (network callback)
11. **LP Sender (non-SWIFT API-to-API)** — `GET /lp-sender/framework` · verify / lock / webhooks / LP redemption. Asset value is recognized by the liquidity-providing bank only while the asset-backed ledger is connected to a regulated FI (`GET /lp-sender/ledger/connection`).

Live docs: `/api/v1/docs` · Spec: `/api/v1/openapi.json`

## Servers

- `https://api.nova-x.io/api/v1` — Public partner API (attach Railway custom domain)
- `https://www.nova-x.io/api/v1` — Public apex (same Nest service)
- `https://api.nova-x.io/api/v1` — Internal Railway origin
- `http://localhost:3000/api/v1` — Local development

## Authentication

- **BearerAuth** — http bearer. From POST /auth/start
- **AdminKey** — apiKey X-Admin-Key.
- **MerchantKey** — apiKey X-Merchant-Key. Also accepts X-Nova-Merchant-Key
- **WebhookSignature** — apiKey X-Nova-Signature.
- **InstitutionApiKey** — apiKey X-Institution-Api-Key. Institution partner API key (hash-stored server-side)
- **MtlsClientCert** — apiKey X-Client-Cert-Fingerprint. mTLS client certificate SHA-256 fingerprint (set by API Gateway/ALB)

228 operations across 40 groups. Required fields are marked with *.

## Accounts

**GET** `/me`

Member profile

Responses: 200 User profile details

---

**GET** `/accounts`

List banking accounts (fiat + crypto)

Parameters: NovaProtocol · Responses: 200 Accounts with IBAN, SWIFT, protocol tier, balances

---

**GET** `/accounts/resolve`

Resolve party by account number or IBAN (never email)

Parameters: accountNumber (query), iban (query), currency (query) · Responses: 200 PartyDestination (kind internal|external)

---

**GET** `/accounts/resolve/me`

Member resolve party by account number or IBAN

Parameters: accountNumber (query), iban (query), currency (query) · Responses: 200 PartyDestination plus isOwn

---

**GET** `/accounts/{accountNumber}`

Lookup account by 4-digit number

Parameters: accountNumber* (path) · Responses: 200 IBAN, SWIFT, currency, protocol

---

**GET** `/wallets`

List all wallets (alias of accounts extended)

Responses: 200 Wallet list

---

**POST** `/wallets/{accountId}/fund`

Sandbox fund (non-production only)

Parameters: accountId* (path) · Body: amount*: string · Responses: 200 Funded (sandbox)

---

## Admin

**POST** `/accounts/admin/ensure-wallet`

Ensure a fiat wallet exists with an explicit 4-digit account number

Body: accountNumber*: string; currency*: string; protocol: online | offline | meta | real | production; userId: string (uuid); email: string (email); label: string · Responses: 200 Existing or newly created wallet; 409 Account number already used with conflicting currency/protocol/user

---

**POST** `/inbound/credit`

Credit Nova account by account number or IBAN

Parameters: IdempotencyKey · Body: accountNumber: string; iban: string; currency*: string; amount*: string; assetClass: fiat | crypto; protocol: online | offline | meta | real | production; reference: string; senderName: string · Responses: 200 Inbound credit posted

---

**GET** `/settlement/ledger/system`

System settlement ledger (admin)

Responses: 200 System float accounts

---

**POST** `/settlement/run`

Run settlement clearing batch

Parameters: IdempotencyKey · Body: kind*: card_clearing | external_clearing; currency*: string · Responses: 201 Batch executed

---

**GET** `/ledger/import/status`

Ledger import middleware status

Responses: 200 Ready flag, paths, supported actions

---

**POST** `/ledger/import/admin`

Bulk ledger import (admin)

Body: LedgerImport (rows*: array of object) · Responses: 200 Import results per row

---

**POST** `/ledger/import/admin/all-to-account`

Import multiple fiats/tokens to one Nova account

Body: accountNumber: string; iban: string; amounts*: array of object; source: string; reference: string · Responses: 200 Batch import results

---

**POST** `/media/admin/upload`

Upload system documentation (admin)

Responses: 200 Upload result with content URL

---

**GET** `/media/admin/list`

List imported documents (admin)

Responses: 200 Document metadata list

---

**POST** `/funding/admin/linked`

Admin seed linked funding for user email

Responses: 200 Seeded

---

## Authentication

**POST** `/auth/start`

Start session (register or login)

Body: StartSession (email*: string (email); pin: string; fullName: string) · Responses: 200 JWT access + refresh tokens, user profile, primary account

---

**GET** `/auth/me`

Current authenticated user

Responses: 200 User object

---

**POST** `/auth/refresh`

Refresh access token

Body: refreshToken*: string · Responses: 200 New token pair

---

**POST** `/auth/logout`

Revoke refresh token

Body: refreshToken*: string · Responses: 204 Logged out

---

## Bank Rail

**GET** `/bank/rail/workflow`

Popular Nova Bank send/receive rail

Responses: 200 Channels, endpoints, dispatch targets

---

**GET** `/bank/rail/ready`

Production readiness for the popular rail

Responses: 200 Ready payload

---

**POST** `/bank/rail/plan`

Plan a send or receive without moving money

Body: direction: send | receive; fromAccountId: string; amount: string; currency: string; toAccountNumber: string; beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string; toAddress: string; destination: string · Responses: 200 Channel, paymentRail, dispatch target. uetr and bankReference are always absent.

---

**POST** `/bank/rail/send`

Send fiat or crypto on the popular rail

Parameters: IdempotencyKey · Body: fromAccountId*: string; amount*: string; currency: string; pin: string; toAccountNumber: string; toAccountId: string (uuid); beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string; toAddress: string; destination: string; network: mainnet | onex | sandbox · Responses: 200 NOVA-RAIL-* payment. UETR only if the bank returned one.

---

**GET** `/bank/rail/receive`

Member IBAN, SWIFT BIC, and crypto deposit addresses

Responses: 200 Fiat coordinates and crypto deposit addresses

---

**GET** `/bank/rail/payments`

List rail payments for the member

Responses: 200 Payment list

---

**GET** `/bank/rail/payments/{id}`

Get a rail payment by UUID or NOVA-RAIL-* id

Parameters: id* (path) · Responses: 200 Payment; 404 Not found

---

**POST** `/bank/rail/payments/{id}/release`

Release a held rail settlement

Parameters: id* (path), IdempotencyKey · Body: authorizationId: string; releaseToken: string · Responses: 200 Released payment, or the existing row when already released; 409 Settlement hash mismatch

---

**POST** `/bank/rail/inbound`

Record inbound credit on the popular rail

Parameters: IdempotencyKey · Body: accountNumber: string; iban: string; currency*: string; amount*: string; assetClass: fiat | crypto; senderName: string; reference: string · Responses: 200 inbound_posted rail payment

## BankPayments

**GET** `/banks/payments/workflow`

Payment orchestrator workflow board and adapter credential readiness

Responses: 200 Workflow board (never invents bank references)

**POST** `/banks/payments`

Create normalized Nova bank payment (dryRun default)

Parameters: Idempotency-Key* (header) · Body: bank*: unionbank | ubs | wells_fargo; rail*: instapay | pesonet | sepa | iban_transfer | ach | wire | swift; amount*: string; currency*: string; sourceAccount*: string; beneficiary*: name*: string; account*: string; bic: string; bankCode: string; m1Available*: string; dryRun: boolean · Responses: 200 Dry-run plan or created draft payment

**POST** `/banks/payments/{id}/submit`

Submit payment to correspondent bank adapter

Parameters: id* (path) · Responses: 200 Submitted with bank-originated reference; 400 Bank reference missing or gate not cleared

**POST** `/banks/payments/{id}/reconcile`

Reconcile against bank status; settle only with bank ref + match

Parameters: id* (path) · Responses: 200 bank_acked or settled

## Cards

**GET** `/cards`

List virtual cards

Responses: 200 Card list (masked)

**POST** `/cards`

Issue virtual Visa card

Body: IssueCard (linkedAccountId*: string (uuid); currency*: string; spendLimit: string; protocol: online | offline; cardholderName: string) · Responses: 201 Card issued

**POST** `/cards/provision-protocol`

Provision online + offline protocol cards

Body: cardholderName: string · Responses: 201 Protocol cards provisioned

---

**POST** `/cards/provision-all`

Provision all fiat cards (USD/EUR/GBP × online/offline)

Body: cardholderName: string · Responses: 201 All fiat wallet cards provisioned

---

**GET** `/cards/{id}/details`

Full card PAN/CVV (authenticated member)

Parameters: id* (path) · Responses: 200 Card credentials for pre-auth

---

**PATCH** `/cards/{id}/status`

Update card status

Parameters: id* (path) · Body: status*: active | frozen | cancelled · Responses: 200 Status updated

---

**POST** `/cards/preauth/initiate`

Visa 101.1 Step 1 — initiate pre-authorization

Parameters: IdempotencyKey · Body: PreAuthInitiate (cardId*: string (uuid); merchant*: string; amount*: string; pan*: string; cvv*: string; expiryMonth*: integer; expiryYear*: integer) · Responses: 201 Pending pre-auth; OTP delivered or returned (sandbox)

---

**POST** `/cards/preauth/{id}/complete`

Visa 101.1 Step 2 — complete with 6-digit OTP

Parameters: id* (path), IdempotencyKey · Body: otp*: string · Responses: 200 Payment approved, ledger posted

---

## Cash

**GET** `/cash/transfers`

List cash transfers

Responses: 200 Cash transfer history

---

**POST** `/cash/deposit`

Cash deposit at branch

Parameters: IdempotencyKey · Responses: 201 Cash deposited

---

**POST** `/cash/withdraw`

Cash withdrawal with pickup code

Parameters: IdempotencyKey · Responses: 201 Withdrawal pending pickup

---

## Chains

**GET** `/chains/status`

Multi-chain registry (Nova One, Anaka, DBIS 138)

Responses: 200 Chain connection status

## Compliance

**GET** `/compliance/status`

Compliance module health (operational)

Responses: 200 Malta compliance stack status

**GET** `/compliance/tier`

KYC tier limits catalog (includes guest tier 0 when unauthenticated)

Responses: 200 Tier limits catalog with member tier when Bearer token sent

**GET** `/compliance/tier/me`

Member KYC tier (guest tier 0 when unauthenticated)

Responses: 200 Member tier and limits, or guest defaults

**GET** `/compliance/cases`

Compliance cases (empty for public; full list for admin JWT)

Responses: 200 Operational status or admin case list

## Crypto

**GET** `/onex/status`

OneX Chain (DBIS 138) status

Responses: 200 RPC, chain ID, wallet mode

**GET** `/onex/chain`

wallet_addEthereumChain parameters

Responses: 200 Chain config for onexwallet

**GET** `/onex/wallet/deposit`

Custodial ETH deposit address

Responses: 200 Deposit address

**GET** `/crypto/dfns/status`

Dfns MPC custody connectivity

Responses: 200 Dfns status

---

**GET** `/crypto/cobo/status`

Cobo WaaS connectivity

Responses: 200 Cobo status

---

**POST** `/onex/wallet/sync`

Sync on-chain deposits to ledger

Responses: 200 Credited deposits

---

**POST** `/onex/wallet/withdraw`

Withdraw ETH to external address

Parameters: IdempotencyKey · Body: accountId*: string (uuid); amount*: string; toAddress*: string · Responses: 201 Withdrawal submitted

---

## Discovery

**GET** `/openapi.json`

OpenAPI specification (this document as JSON)

Responses: 200 OpenAPI 3.0 document

---

**GET** `/docs`

Swagger UI

Responses: 200 Interactive API explorer (HTML)

---

**GET** `/global/status`

Global platform status

Responses: 200 Environment, protocol, provider matrix, rail URLs

---

**GET** `/global/integrations`

Integration PASS/FAIL report

Responses: 200 Visa, SWIFT, Fineract, OneX, production UX checks

---

**GET** `/international/integrations`

International banks, platforms, crypto & trading catalog

Parameters: domain (query) · Responses: 200 Catalog with readiness flags and corridors

---

**GET** `/international/corridors`

International trade corridors with ranked rails

Responses: 200 Corridor matrix

---

**GET** `/global/stats`

Platform statistics

---

**GET** `/global/recent-activity`

Recent platform activity feed

Parameters: LimitQuery

---

## External Rails

**GET** `/external/transfers`

List external inbound/outbound transfers

Responses: 200 External transfer history

---

**GET** `/external/resolve/{iban}`

Resolve Nova account by IBAN

Parameters: iban* (path) · Responses: 200 Nova account metadata if IBAN is on-file; 404 Unknown IBAN

---

**POST** `/external/deposit`

Register inbound external deposit (sandbox)

Parameters: IdempotencyKey · Body: ExternalDeposit (accountId*: string (uuid); amount*: string; senderIban: string; senderSwift: string; senderName: string; reference: string) · Responses: 201 Inbound credited to online ledger

---

**POST** `/external/transfer`

Outbound SWIFT/SEPA to external IBAN

Parameters: IdempotencyKey, NovaProtocol · Body: ExternalOutbound (fromAccountId*: string (uuid); amount*: string; beneficiaryIban*: string; beneficiarySwift*: string; beneficiaryName*: string; pin: string; reference: string) · Responses: 201 Outbound transfer posted

---

## Fiat

**POST** `/fiat/send`

Send fiat from a Nova Bank account

Parameters: IdempotencyKey, NovaProtocol · Body: FiatSend (fromAccountId*: string; amount*: string; currency: string; toAccountNumber: string; toAccountId: string (uuid); beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string; pin: string; reference: string; rail: auto | ledger | bank) · Responses: 200 Fiat send posted to the ledger (and bank adapter when external); 400 Invalid destination or insufficient balance

---

**GET** `/fiat/sends`

List recent fiat sends

Responses: 200 Ledger transfers plus outbound bank rails

---

**GET** `/fiat/send/{id}`

Get a fiat send

Parameters: id* (path) · Responses: 200 Ledger or bank payment; 404 Not found

---

**GET** `/fiat/send/{id}/status`

Fiat send status

Parameters: id* (path) · Responses: 200 Status snapshot

---

**GET** `/fiat/attestation/workflow`

Fiat Attestation + On-Chain Proof model

Responses: 200 Layers, checks, and endpoints

---

**POST** `/fiat/attestation`

Issue a Nova fiat attestation

Parameters: IdempotencyKey · Body: fromAccountId*: string (uuid); amount*: string; currency*: string; fiatReference*: string; fiatProofKind*: bank_statement | emi_api | swift_confirmation | nostro_balance | ledger_only; treasuryAccount: string; beneficiary: string; dryRun: boolean · Responses: 200 Dry-run plan or persisted NOVA-FIAT-* attestation

---

**GET** `/fiat/attestation/{id}`

Public attestation document for counterparty banks

Parameters: id* (path) · Responses: 200 Attestation without claiming legally recognized bank money

---

**GET** `/fiat/attestation/{id}/verify`

Verify attestation hash and signature

Parameters: id* (path) · Responses: 200 valid, hashOk, signatureOk, status

---

**GET** `/fiat/attestation/{id}/package`

Three-proof package (fiat + Nova attestation + optional chain)

Parameters: id* (path) · Responses: 200 Counterparty verification package

---

**POST** `/fiat/attestation/{id}/link-onchain`

Attach optional blockchain proof after a real crypto purchase

Parameters: IdempotencyKey · Body: txHash: string; settlementId: string; purchaseId: string; confirmations: integer; onchainBalance: string; dryRun: boolean · Responses: 200 On-chain proof plan or updated package

---

## Fineract

**GET** `/fineract/status`

Apache Fineract gateway status

Responses: 200 Tenant, API mode, external swagger URL

**GET** `/fineract/connection/test`

Fineract connectivity test

Responses: 200 Gateway and API live flags

## Funding

**GET** `/funding/linked`

List linked external funding sources

Responses: 200 IBAN + card on file per currency

**POST** `/funding/linked`

Upsert linked external bank + card

Body: LinkedFunding (currency*: string; beneficiaryName*: string; iban*: string; swiftBic*: string; bankName: string; bankAddress: string; cardPan: string; cardCvv: string; cardNetwork: string; cardExpiryMonth: integer; cardExpiryYear: integer; label: string; protocol: string) · Responses: 200 Linked source saved

**GET** `/funding/linked/{currency}/card`

Linked card details for currency

Parameters: currency* (path) · Responses: 200 Masked card metadata

## Health

**GET** `/health/live`

Liveness probe

Responses: 200 API process is up

**GET** `/health/ready`

Readiness probe (database)

Responses: 200 Ready to serve traffic

**GET** `/health`

Full health check

Responses: 200 Database (+ Redis if configured)

## Institution

**GET** `/institutional/status`

Institution API health and linked client context

Responses: 200 Institution status; 401 Missing/invalid API key or fingerprint mismatch

---

**GET** `/institutional/balances`

Ledger balances for the linked primary member

Responses: 200 Account balances; 401 Auth failure; 403 Missing balances:read scope

---

**GET** `/institutional/wires`

List outbound external/partner wires for the linked member

Parameters: LimitQuery · Responses: 200 Wire list; 401 Auth failure; 403 Missing wires:read scope

---

**POST** `/institutional/wires`

Initiate outbound wire via external banking rail

Parameters: IdempotencyKey · Body: fromAccountId*: string (uuid); amount*: string; beneficiaryIban*: string; beneficiarySwift*: string; beneficiaryName*: string; pin*: string; reference: string; bankName: string; bankCountry: string; dryRun: boolean · Responses: 200 Dry-run result or initiated transfer; 401 Auth failure; 403 Missing wires:write scope

---

**POST** `/institutional/inbound/credit`

Inbound settlement credit into an allowlisted Nova account

Parameters: IdempotencyKey · Body: amount*: string; currency*: string; accountNumber*: string; settlementReference: string; reference: string; senderName: string; protocol: online | offline | meta | real | production; dryRun: boolean · Responses: 200 Dry-run or completed inbound credit evidence; 401 Auth failure; 403 Missing inbound:credit scope or destination/currency not allowlisted; 404 Destination Nova account wallet not found

---

**GET** `/institutional/lp/ping`

LP Sender ledger connectivity ping

Responses: 200 Ledger ping; 401 Missing/invalid API key or fingerprint mismatch

---

**POST** `/institutional/lp/verify`

Verify LP source asset (Nova account 6572 USD)

Body: accountNumber: string; currency: string; amount: string · Responses: 200 Asset verification; 401 Auth failure; 404 Wallet not found

---

**POST** `/institutional/lp/lock`

LP lock (dry-run by default)

Parameters: IdempotencyKey · Body: amount*: string; accountNumber: string; currency: string; assetRef: string; dryRun: boolean · Responses: 200 Lock dry-run recorded; 400 Live lock gated or invalid amount; 401 Auth failure

---

**POST** `/institutional/lp/redeem`

LP redeem (dry-run by default)

Parameters: IdempotencyKey · Body: lockId: string (uuid); amount: string; currency: string; dryRun: boolean · Responses: 200 Redeem dry-run recorded; 400 Live redeem gated; 401 Auth failure

---

**POST** `/institutional/lp/webhooks`

Register LP HTTPS webhook callback

Body: callbackUrl*: string (uri) · Responses: 200 Webhook registered; 400 callbackUrl must be HTTPS; 401 Auth failure

---

**POST** `/institutional/lp/webhooks/{id}/test`

Send a signed LP webhook test event

Parameters: id* (path) · Responses: 200 Test event attempted (delivery HTTP status recorded); 401 Auth failure; 404 Webhook not found

---

**GET** `/institutional/clients`

List institution clients (super_admin)

Responses: 200 Institution client rows

---

**POST** `/institutional/clients`

Upsert institution client and optionally link API key (super_admin)

Body: institutionName*: string; legalName: string; domain: string; mtlsCertFingerprint: string; primaryUserId: string (uuid); apiKeyId: string (uuid); status: active | suspended | revoked; metadata: object · Responses: 200 Upserted institution client

---

**POST** `/institutional/api-keys`

Create institution API key (super_admin)

Body: institutionName*: string; mtlsCertFingerprint: string; institutionClientId: string (uuid); ipAllowlist: array of string; scopes: array of string · Responses: 201 API key created (plaintext returned once)

---

**POST** `/institutional/api-keys/{id}/scopes`

Grant additional scopes on an institution API key (super_admin)

Parameters: id* (path) · Body: scopes*: array of string · Responses: 200 Updated key scopes; 404 API key not found

---

## International

**POST** `/international/route`

Route an international trade to the best available rail

Body: kind*: fiat_international | fiat_sepa | crypto_spot | crypto_withdraw | dex_swap | documentary_credit | standby_guarantee | commodity_rwa; currency: string; assetClass: fiat | crypto | rwa; preferPartner: string · Responses: 200 Selected rail, alternatives, nextSteps (execute fiat via /international/send)

---

**GET** `/international/send/workflow`

International IBAN + SWIFT send product

Responses: 200 Endpoints, networks (sepa|swift), no-invent rule

---

**GET** `/international/send/ready`

Production readiness for international IBAN/SWIFT send

Responses: 200 Ready payload

---

**POST** `/international/send/plan`

Plan SEPA vs SWIFT for IBAN+BIC without moving money

Body: fromAccountId: string; amount: string; currency: string; beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string; reference: string · Responses: 200 network sepa|swift, dispatch fiat_send. uetr always absent.

---

**POST** `/international/send`

Send fiat internationally by IBAN + BIC (SEPA or SWIFT)

Parameters: IdempotencyKey · Body: fromAccountId*: string; amount*: string; currency: string; beneficiaryIban*: string; beneficiarySwift*: string; beneficiaryName*: string; reference: string; pin*: string · Responses: 200 Payment posted. path=/international/send; underlyingPath=/fiat/send

---

## LP Sender

**GET** `/lp-sender/framework`

Operator hand-off — answers integration requirements 01–04

Responses: 200 Framework, endpoints, webhook policy, developer hand-off

---

**GET** `/lp-sender/ledger/connection`

Regulated-FI connection that makes asset value recognizable

Parameters: refresh (query) · Responses: 200 Connection snapshot

---

**GET** `/lp-sender/assets`

01 — List verifiable assets (existence, ownership, instrument details)

Responses: 200 Assets for the institution primary member; 401 Missing/invalid API key or fingerprint; 403 Missing lp:verify (or balances:read)

---

**GET** `/lp-sender/assets/{id}`

01 — Verify a single asset

Parameters: id* (path) · Responses: 200 Asset exists and ownership verified; 404 Asset not found for this institution

---

**GET** `/lp-sender/transactions/{id}`

01 — Verify a ledger transfer

Parameters: id* (path) · Responses: 200 exists true/false plus ownership and instrument fields; 401 Auth failure

---

**POST** `/lp-sender/verify`

01 — Verify by assetId, transactionId, or accountNumber

Parameters: IdempotencyKey · Body: assetId: string (uuid); transactionId: string (uuid); accountNumber: string · Responses: 200 Verification result; 400 No identifier provided; 404 Asset not found

---

**POST** `/lp-sender/locks`

02 — Reserve / lock asset against bank issuance

Parameters: IdempotencyKey · Body: assetAccountId*: string (uuid); amount*: string; releaseCondition: ttl | manual_release | lp_bank_request | on_redemption_settled; expiresAt: string (date-time); dryRun: boolean · Responses: 200 Lock held (or dry-run); 400 Insufficient balance or invalid amount; 409 LEDGER_NOT_BANK_CONNECTED — LP bank will not recognize the value

---

**GET** `/lp-sender/locks/{id}`

02 — Get lock confirmation, timestamps, and release conditions

Parameters: id* (path) · Responses: 200 Lock record; 404 Lock not found

---

**POST** `/lp-sender/locks/{id}/release`

02 — Release a held lock (reverses the ledger hold)

Parameters: id* (path), IdempotencyKey · Body: condition: ttl | manual_release | lp_bank_request | on_redemption_settled; dryRun: boolean · Responses: 200 Lock released; 409 Lock not in held state, or ledger not bank-connected

---

**GET** `/lp-sender/webhooks`

03 — List status callback subscriptions

Responses: 200 Webhook subscriptions (signing secrets never listed)

---

**POST** `/lp-sender/webhooks`

03 — Register HTTPS callback URL (signing secret shown once)

Parameters: IdempotencyKey · Body: callbackUrl*: string (uri); eventTypes: array of string · Responses: 200 Subscription created; signingSecret returned once; 400 Callback URL not https or host not allowed

---

**POST** `/lp-sender/webhooks/{id}/rotate`

03 — Rotate webhook signing secret

Parameters: id* (path), IdempotencyKey · Responses: 200 New signing secret (shown once)

---

**POST** `/lp-sender/webhooks/{id}/test`

03 — Deliver a signed test event

Parameters: id* (path) · Responses: 200 Delivery attempt result

---

**POST** `/lp-sender/redemptions`

04 — Initiate LP redemption (API-to-API, non-SWIFT)

Parameters: IdempotencyKey · Body: lockId: string (uuid); amount*: string; currency*: string; payrollParty: nova_institution_primary | lp_bank | named; payrollPartyName: string; destination: string; settlementInstructions: object; dryRun: boolean · Responses: 200 Redemption initiated or dry-run; 409 LEDGER_NOT_BANK_CONNECTED

---

**GET** `/lp-sender/redemptions/{id}`

04 — Redemption status, confirmation, post-settlement transfer id

Parameters: id* (path) · Responses: 200 Redemption record; 404 Not found

---

**POST** `/lp-sender/redemptions/{id}/settle`

04 — Post-settlement transfer (ledger) after LP confirmation

Parameters: id* (path), IdempotencyKey · Body: dryRun: boolean · Responses: 200 Settled or dry-run; 409 Cannot settle from current status, or ledger not bank-connected

---

## Ledger Bridge

**GET** `/ledger-bridge/sources`

PC ledger source definitions

Responses: 200 ledger-pro, local-ledger-2026, crypto-ledger

---

**GET** `/ledger-bridge/status`

Balances grouped by ledger source

Responses: 200 Per-source balances and available actions

---

**POST** `/ledger-bridge/process`

Execute multi-row settlement workflow

Parameters: IdempotencyKey, NovaProtocol · Body: LedgerBridgeProcess (pin: string; rows*: array of WorkflowRow) · Responses: 200 Per-row results and updated balances

---

**POST** `/ledger-bridge/settle`

Cross-ledger settlement preset

Parameters: IdempotencyKey · Body: LedgerBridgeSettle (pin: string; includeExternal: boolean; beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string) · Responses: 200 Settlement workflow results

---

## Ledger Send

**GET** `/ledger-send/orders`

List ledger send orders

Parameters: LimitQuery · Responses: 200 Ledger send history (to_local and to_online)

---

**POST** `/ledger-send/orders`

Send between online Ledger and Local Ledger

Parameters: IdempotencyKey · Body: fromAccountId*: string (uuid); amount*: string; currency*: string; direction: to_local | to_online · Responses: 201 Order completed

---

**POST** `/ledger-send/to-online`

Make online from Local Ledger (m1 → m0)

Parameters: IdempotencyKey · Body: fromAccountId*: string (uuid); amount*: string; currency*: string · Responses: 201 Offline fiat moved to the paired online Ledger

---

**POST** `/ledger-send/convert`

Convert local ledger to real-world payout

Parameters: IdempotencyKey · Body: LedgerSendConvert (fromAccountId*: string (uuid); amount*: string; kind*: bank_transfer | cash_withdrawal | goods_services | real_assets; beneficiaryIban: string; beneficiarySwift: string; beneficiaryName: string; pin: string; reference: string) · Responses: 201 Conversion initiated

---

## Media

**GET** `/media/status`

System documentation import status

Responses: 200 Ready flag, allowed extensions, stored count

---

**GET** `/media/content`

Primary how-it-works documentation (public)

Responses: 200 Markdown or plain-text system guide

---

**GET** `/media/content/{id}`

Specific imported document by id

Parameters: id* (path) · Responses: 200 Document content

---

## Merchant

**POST** `/cards/merchant/preauth/initiate`

Merchant-initiated pre-auth (Step 1)

Parameters: IdempotencyKey · Body: PreAuthInitiate (cardId*: string (uuid); merchant*: string; amount*: string; pan*: string; cvv*: string; expiryMonth*: integer; expiryYear*: integer) · Responses: 201 Pending — cardholder completes OTP

---

**GET** `/cards/merchant/preauth/{preAuthId}`

Poll merchant pre-auth status

Parameters: preAuthId* (path) · Responses: 200 pending | completed | expired | failed

---

**POST** `/cards/merchant/preauth/{preAuthId}/complete`

Merchant completes pre-auth with OTP

Parameters: IdempotencyKey · Body: otp*: string · Responses: 200 Approved

---

**POST** `/cards/{id}/authorize`

Direct card authorization (merchant)

Parameters: id* (path), IdempotencyKey · Body: merchant*: string; amount*: string · Responses: 200 Authorization result

---

## Nova Chain

**GET** `/nova-chain/status`

Nova One chain RPC status (port 22016)

Responses: 200 Chain status

---

**GET** `/nova-chain/tokens`

Token catalog on Nova One chain

Responses: 200 Token list

---

**POST** `/nova-chain/tokens/transfer`

Transfer tokens on Nova One via Marionette

Parameters: IdempotencyKey · Responses: 201 On-chain token transfer

---

## NovaCryptoHost

**GET** `/integrations/alshumookh/status`

ALSHUMOOKH production readiness without exposing credentials

Responses: 200 Boolean API key, HMAC, receiver, and live-mode readiness

---

**POST** `/integrations/alshumookh/m1-usd`

Submit evidence for an existing $1,000 USD M1 on-chain transfer to ALSHUMOOKH

Parameters: IdempotencyKey · Body: transactionReference*: string; txHash*: string; senderWallet*: string; receiverWallet*: string; amount*: string; network*: ethereum | base | tron-placeholder; tokenContract*: string; timestamp*: string (date-time); authorizationCode: string; dryRun: boolean · Responses: 200 Dry-run payload or ALSHUMOOKH ingestion response

---

**GET** `/nova-crypto-host/status`

Nova Crypto Host product status (wallets, mode, master→master production rail, Nova peer)

Responses: 200 Host status

---

**GET** `/nova-crypto-host/ledgers`

Unified off/on-ledger, custody, and on-chain snapshot

Parameters: userId (query) · Responses: 200 Ledger entries + wallet snapshots

---

**POST** `/nova-crypto-host/import/hybx`

Import Hybx Fineract deposits and promote to m4

Parameters: IdempotencyKey · Body: email*: string (email); fineractTransactionId: string; currency: string; promoteToM4: boolean; dryRun: boolean · Responses: 200 Import job result

---

**POST** `/nova-crypto-host/import/nsb`

Import NSB AUSDT surplus into Nova LP float

Parameters: IdempotencyKey · Body: reference*: string; pool: reserve | insurance | surplus; fromAzureSurplus: boolean; dryRun: boolean · Responses: 200 NSB import result

---

**POST** `/nova-crypto-host/import/pc-ledgers`

Import fiats and tokens from Ledger Pro, Local Ledger 2026, Crypto Ledger

Parameters: IdempotencyKey · Body: email*: string (email); sources: value; useBundledSources: boolean; importOnly: boolean; promoteToM4: boolean; dryRun: boolean; rows: array of object · Responses: 200 PC ledger import job result

---

**POST** `/nova-crypto-host/settle`

Convert to real fiat (m3/m4) and/or fiat→USDT ERC-20 treasury / master payout

Parameters: IdempotencyKey · Body: email*: string (email); target*: real_fiat | real_crypto | both; walletClass: master | treasury | custody | recipient_0x | nova_account; amount: string; currency: string; fromAccountId: string; recipientAddress: string; dryRun: boolean · Responses: 200 Settlement job result

---

**POST** `/nova-crypto-host/payout`

Master→master api_crypto_host package create or settle-onchain (production)

Parameters: IdempotencyKey · Body: packageId: string (uuid); create: object; userId*: string; fromAccountId*: string; settleOnchain: boolean; dryRun: boolean · Responses: 200 Payout job result

---

**POST** `/nova-crypto-host/settlement/hmac-external`

Release existing m4 USD through m1 and transfer it to another Nova HMAC peer

Parameters: IdempotencyKey, X-Settlement-Approval (header) · Body: senderEmail*: string (email); recipientEmail*: string (email); amount*: string; currency*: USD; reference*: string; approvalReference: string; dryRun: boolean · Responses: 200 Dry-run plan or completed HMAC peer settlement

---

**POST** `/nova-crypto-host/nova/sync`

Sync with Nova crypto-host peer (push/pull/bidirectional)

Parameters: IdempotencyKey · Body: direction*: push | pull | bidirectional; settleTarget: real_fiat | real_crypto | both; email: string (email); dryRun: boolean · Responses: 200 Nova peer sync result

---

**POST** `/nova-crypto-host/nova/webhook`

Inbound Nova peer webhook (HMAC + API key)

Parameters: X-Nova-Api-Key* (header), X-Nova-Timestamp* (header), X-Nova-Signature* (header) · Body: eventType*: string; eventId*: string; occurredAt*: string; payload*: object · Responses: 200 Webhook accepted

## Partners

**GET** `/partners/status`

Wise and Revolut integration status

Responses: 200 Partner status

**POST** `/partners/wise/quote`

Wise FX quote

Responses: 200 Quote

**POST** `/partners/wise/transfer`

Send via Wise

Parameters: IdempotencyKey · Responses: 201 Wise transfer initiated

**POST** `/partners/revolut/quote`

Revolut FX quote

Responses: 200 Quote

**POST** `/partners/revolut/transfer`

Send via Revolut Business API

Parameters: IdempotencyKey · Responses: 201 Revolut transfer posted

## Protocol

**GET** `/protocol/capabilities`

Nova Sync 101.1 capabilities

Responses: 200 Supported operations, modules m0/m1, idempotency

**GET** `/protocol/status`

Protocol status

**GET** `/servers/peers`

Online/offline peer servers

## SWIFT

**GET** `/swift/status`

SWIFT network integration status

Responses: 200 OAuth, sandbox, Nova BIC, GPI paths

---

**GET** `/swift/oauth/test`

Test SWIFT.com OAuth connection

Responses: 200 OAuth test result

---

**POST** `/swift/validate`

Payment pre-validation (SWIFT.com or local rules)

Body: SwiftValidate (amount*: string; currency*: string; beneficiaryIban*: string; beneficiarySwift*: string; beneficiaryName*: string) · Responses: 200 Validation checks and UETR

---

**GET** `/swift/track/{uetr}`

GPI payment tracking by UETR

Parameters: uetr* (path) · Responses: 200 GPI events (never simulated ACCC credit)

---

**GET** `/swift/gpi/production`

Production SWIFT gpi package checklist

Responses: 200 Production package items, flow, states, blockers

---

## SWIFT Messages

**GET** `/swift/messages/catalog`

Worldwide SWIFT MT catalog with formal verbiage

Parameters: category (query) · Responses: 200 MT catalog entries and buildable list

---

**GET** `/swift/messages/rwa-verbiage`

RWA formal verbiage mapped to recommended MT types

Parameters: kind (query) · Responses: 200 RWA verbiage blocks

---

**POST** `/swift/messages/mt103`

Generate MT103 single customer credit transfer

Responses: 201 MT103 message generated

---

**POST** `/swift/messages/mt102`

Generate MT102 multiple customer credit transfer

Responses: 201 MT102 message generated

---

**POST** `/swift/messages/mt600`

Generate MT600 commodity trade confirmation (RWA)

Responses: 201 MT600 message generated

---

**POST** `/swift/messages/mt700`

Generate MT700 issue of documentary credit (RWA LC)

Responses: 201 MT700 message generated

---

**POST** `/swift/messages/mt710`

Generate MT710 advice of third bank documentary credit

Responses: 201 MT710 message generated

---

**POST** `/swift/messages/mt760`

Generate MT760 guarantee / standby letter of credit

Responses: 201 MT760 message generated

---

**POST** `/swift/messages/mt541`

Generate MT541 receive against payment (RWA DvP)

Responses: 201 MT541 message generated

---

**POST** `/swift/messages/mt543`

Generate MT543 deliver against payment (RWA DvP)

Responses: 201 MT543 message generated

---

## Settlement

**GET** `/settlement/summary`

Settlement summary (unsettled items)

Responses: 200 Card auths, external transfers, system floats

---

**GET** `/settlement/ledger/mine`

Member settlement ledger entries

Parameters: LimitQuery · Responses: 200 Ledger entries

---

**GET** `/settlement/batches`

List settlement batches

Parameters: LimitQuery · Responses: 200 Batch history

---

**GET** `/settlement/activity`

Unified settlement activity

Parameters: LimitQuery · Responses: 200 Transfers, cards, external, ledger_send

## Swap

**GET** `/swap/markets`

List swap markets

Responses: 200 Market definitions

**GET** `/swap/status`

Nova Swap / Marionette status

Responses: 200 Provider connectivity

**GET** `/swap/balances`

Swap balances for member

Responses: 200 GL balances

**POST** `/swap/wallets/ensure`

Create customer fiat books (m1–m4) and popular crypto vaults

Parameters: IdempotencyKey · Responses: 200 Funding wallets plus spot balances. inventedLiquidity is always false.

**GET** `/swap/orderbook`

Order book for market

Parameters: market* (query) · Responses: 200 Bids and asks

**POST** `/swap/quote`

Get swap quote

Body: SwapQuote (marketId*: string; side*: buy | sell; amount*: string; slippageBps: number) · Responses: 200 Quote with price impact

**POST** `/swap/execute`

Execute swap

Parameters: IdempotencyKey · Body: SwapQuote (marketId*: string; side*: buy | sell; amount*: string; slippageBps: number) · Responses: 201 Swap executed

**GET** `/swap/history`

Swap history

Parameters: LimitQuery · Responses: 200 Past swaps

## Sync

**POST** `/sync/push`

Push offline operation queue

Parameters: NovaProtocol · Body: SyncPush (deviceId*: string; operations*: array of object) · Responses: 200 Applied operations

**GET** `/sync/pull`

Pull server state since cursor

Parameters: since (query), deviceId (query) · Responses: 200 Pending operations and cursor

## Tracker

**GET** `/tracker/overview`

Counts by bank/status plus nostro summary

Parameters: SwiftOrAdminKey · Responses: 200 Overview metrics

**GET** `/tracker/banks`

Nova / Anaka / zBank readiness and SWIFT OAuth

Parameters: SwiftOrAdminKey · Responses: 200 Bank readiness

**GET** `/tracker/transfers`

Unified transfer list across rails

Parameters: SwiftOrAdminKey, bank (query), rail (query), status (query), iban (query), uetr (query), q (query), limit (query), offset (query) · Responses: 200 Paginated unified transfers

**GET** `/tracker/transfers/{id}`

Canonical transfer detail with timeline

Parameters: SwiftOrAdminKey, id* (path) · Responses: 200 Transfer + events

**GET** `/tracker/uetr/{uetr}`

Resolve transfer and live/sim GPI by UETR

Parameters: SwiftOrAdminKey, uetr* (path) · Responses: 200 Transfer + GPI track + persisted events

**POST** `/tracker/refresh/{uetr}`

Force GPI refresh and rewrite events

Parameters: SwiftOrAdminKey, uetr* (path), Idempotency-Key* (header) · Responses: 200 Refreshed transfer + GPI

**GET** `/tracker/iban/{iban}`

Find transfers by IBAN across rails

Parameters: SwiftOrAdminKey, iban* (path) · Responses: 200 Matching transfers

**POST** `/tracker/ingest`

Idempotent ingest for Anaka/zBank-linked MT/UETR

Parameters: SwiftOrAdminKey, Idempotency-Key* (header) · Body: bank*: nova | anaka | zbank; rail: wire | swift_external | anaka | zbank | swift_gpi; amount: string; currency: string; uetr: string (uuid); iban: string; bic: string; debtorName: string; creditorName: string; reference: string; status: string; sourceId: string; mt103Preview: string · Responses: 200 Ingested canonical transfer

---

**POST** `/tracker/production`

Full production register (API key + amount + SWIFT + reference + data)

Parameters: SwiftOrAdminKey, Idempotency-Key* (header) · Body: bank: nova | anaka | zbank; amount*: string; currency*: string; referenceNumber*: string; swiftCode*: string; data*: value; creditorName: string; iban: string; uetr: string (uuid) · Responses: 200 Production transfer registered with timeline

---

**GET** `/tracker/reference/{reference}`

Lookup transfers by reference number (optional amount + SWIFT filters)

Parameters: SwiftOrAdminKey, reference* (path), amount (query), swiftCode (query) · Responses: 200 Matching transfers

---

## Trade Finance

**GET** `/trade-finance/types`

List trade finance instrument types (LC, SBLC, DLC, BG)

Responses: 200 Instrument types

---

**GET** `/trade-finance/instruments`

List trade finance instruments

Responses: 200 Instrument list

---

**POST** `/trade-finance/instruments`

Issue LC, SBLC, DLC, or Bank Guarantee

Parameters: IdempotencyKey · Responses: 201 Instrument issued

---

## Transactions

**GET** `/transactions`

Account transaction history

Parameters: LimitQuery, offset (query) · Responses: 200 Paginated transactions

---

**GET** `/transactions/cards`

Card transaction history

Parameters: LimitQuery · Responses: 200 Card spends

---

## Transfers

**POST** `/transfers`

Transfer between two accounts

Parameters: IdempotencyKey, NovaProtocol · Body: Transfer (fromAccountId*: string (uuid); toAccountId*: string (uuid); amount*: string; currency*: string; reference: string) · Responses: 201 Transfer completed

---

**POST** `/transfers/by-account`

Transfer to 4-digit account number

Parameters: IdempotencyKey · Body: TransferByAccount (fromAccountId*: string (uuid); toAccountNumber*: string; amount*: string; pin: string; reference: string) · Responses: 201 Transfer completed

---

**POST** `/transfers/by-party`

Transfer by account number or IBAN+BIC (never email)

Parameters: IdempotencyKey · Body: fromAccountId*: string (uuid); toAccountNumber: string; toIban: string; toSwift: string; beneficiaryName: string; amount*: string; pin: string; reference: string · Responses: 200 Transfer result with rail=internal|external

---

**POST** `/transfers/exchange`

FX exchange between accounts

Parameters: IdempotencyKey · Body: Exchange (fromAccountId*: string (uuid); toAccountId*: string (uuid); fromAmount*: string) · Responses: 201 Exchange completed

---

**GET** `/transfers/convert-to-stable`

Quote fiat → USDC/USDT against treasury inventory (not Exchange books)

Parameters: fromAccountId* (query), amount* (query), stable* (query) · Responses: 200 Quote or blocked plan (wouldSucceed false when coverage is short)

---

**POST** `/transfers/convert-to-stable`

Convert fiat → USDC/USDT. Defaults to dry-run; live needs dryRun false

Parameters: IdempotencyKey · Body: fromAccountId*: string; amount*: string; stable*: USDC | USDT; dryRun: boolean · Responses: 200 Dry-run quote, blocked plan, or completed convert; 400 Live convert blocked (coverage, balance, or missing vault)

---

**GET** `/transfers/buy-crypto`

Quote tokenized fiat → popular crypto against treasury inventory (not Exchange books)

Parameters: fromAccountId* (query), amount* (query), asset* (query) · Responses: 200 Quote or blocked plan (wouldSucceed false when coverage is short)

---

**POST** `/transfers/buy-crypto`

Buy popular crypto with tokenized fiat. Defaults to dry-run; live needs dryRun false

Parameters: IdempotencyKey · Body: fromAccountId*: string; amount*: string; asset*: BTC | ETH | SOL | USDC | USDT | BNB | TRX; dryRun: boolean · Responses: 200 Dry-run quote, blocked plan, or completed buy; 400 Live buy blocked (coverage, balance, or missing vault)

## Treasury

**GET** `/treasury/crypto/workflow`

Treasury crypto settlement evidence chain

Responses: 200 Workflow board

**POST** `/treasury/crypto/purchase`

Authorize an M1-funded treasury crypto purchase

Parameters: IdempotencyKey · Body: sourceCurrency*: string; sourceAmount*: string; asset*: USDT | ETH; network*: string; purpose: string; idempotencyKey: string; fromAccountId: string (uuid); dryRun: boolean · Responses: 200 Purchase authorized (or dry-run preview); 400 M1 not available or safety check failed

**GET** `/treasury/crypto/purchase/{id}`

Get a treasury crypto purchase

Parameters: id* (path) · Responses: 200 Purchase record; 404 Not found

**POST** `/treasury/crypto/settlement`

Open a crypto settlement after the actual LP/OTC purchase

Parameters: IdempotencyKey · Body: purchaseId*: string; asset*: USDT | ETH; network*: string; amount*: string; destination: string; custodyProvider: string; dfnsWalletId: string; txHash: string; dryRun: boolean · Responses: 200 Settlement opened at pending_settlement; 400 Ethereum safety check failed

**GET** `/treasury/crypto/settlement/{id}`

Get a crypto settlement

Parameters: id* (path) · Responses: 200 Settlement FSM row; 404 Not found

**POST** `/treasury/crypto/settlement/{id}/advance`

Advance settlement from a real Ethereum transaction

Body: txHash: string · Responses: 200 Updated FSM row

**GET** `/treasury/crypto/balance`

Fresh DFNS on-chain USDT and native ETH

Responses: 200 On-chain balances

**GET** `/treasury/crypto/reconciliation/{id}`

Reconcile expected vs on-chain vs crypto ledger

Parameters: id* (path) · Responses: 200 Three-way reconciliation row; 404 Not found

## Visa

**GET** `/visa/status`

VisaNet / Marqeta integration status

Responses: 200 JIT URL, sandbox mode, pre-auth info

**POST** `/visa/marqeta/jit-funding`

Marqeta JIT funding webhook (Visa network)

Parameters: authorization (header) · Body: object · Responses: 200 JIT funding decision

## Webhooks

**POST** `/webhooks/{provider}`

Generic signed webhook ingress

Parameters: provider* (path) · Body: eventType*: string; payload*: object · Responses: 200 Accepted

**POST** `/webhooks/{provider}/card-auth`

Card authorization webhook

Parameters: provider* (path) · Body: object · Responses: 200 Processed

## Wire

**GET** `/wire/transfers`

List wire transfers

Responses: 200 Wire transfer history

**POST** `/wire/send`

Send international wire (MT103 + GPI)

Parameters: IdempotencyKey, NovaProtocol · Body: fromAccountId*: string; amount*: string; beneficiaryName*: string; beneficiaryIban: string; beneficiarySwift: string; chargeBearer: OUR | SHA | BEN; pin: string · Responses: 201 Wire sent with MT103 and UETR

**GET** `/wire/gpi/payments`

List GPI-tracked wire payments

Responses: 200 GPI payment list with UETR